

Mini SNMP Agent

Arturo Francés y Marcos Fraile

1. RESUMEN:

En este informe se describe el desarrollo e implementación de una mib y un gestor creados por nosotros. Además, realiza una valoración del desarrollo del diseño, destacando los aspectos más relevantes y las decisiones adoptadas.

https://github.com/marcosf22/SNMP_mini_Agent

2. DISEÑO DEL SISTEMA:

Para realizar el agente SNMP personalizado, hemos seguido un orden estructurado que abarca desde la conceptualización de la información (el diagrama UML) hasta su implementación en Python ayudándonos de diferentes LLMs como lo son Gemini y/o ChatGPT.

El objetivo central del proyecto fue desarrollar un agente completo, dotado de su propia Base de Información de Gestión (MIB), capaz de exponer métricas de rendimiento y parámetros de configuración, así como de emitir alertas cuando se superan umbrales predefinidos.

En los siguientes puntos se analizarán más en detalle los pasos seguidos para llevar a cabo el agente y su monitorización.

2.1 Diagrama UML

En primer lugar, hemos definido un diagrama UML que nos permite establecer la estructura de los objetos a gestionar, sus atributos y las relaciones entre ellos, con independencia de la sintaxis específica del SNMP. Para llevar a cabo esta tarea, hicimos un pequeño boceto a mano de como debería de quedar el diagrama y posteriormente le preguntamos a Gemini como haría el diagrama que se nos pedía. Lo que nos dió fue un código que era legible por la web: <https://www.mermaidchart.com/> donde pegamos el código y se generó el diagrama UML que la IA había planteado. Tras verificar nuestro boceto con el diagrama hecho por Gemini, vimos que, efectivamente, el boceto realizado era correcto.

En este caso, el diagrama UML nos muestra cómo se supervisa el uso de la CPU de un sistema y cómo se almacena la información del administrador responsable de recibir las notificaciones.

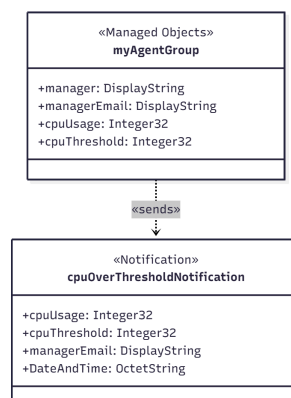


Figura 1: Diagrama UML con los objetos gestionados

Los elementos que aparecen el diagrama UML son:

- **Objetos Gestionados:**

manager (DisplayString): Almacena el nombre del administrador de red responsable.

managerEmail (DisplayString): Contiene la dirección de correo electrónico del administrador para el envío de notificaciones.

cpuUsage (Integer32): Representa el uso actual de la CPU como un valor porcentual.

cpuThreshold (Integer32): Define el umbral de uso de CPU que, al ser superado, desencadena una notificación.

- **Notificación:**

cpuOverThresholdNotification: Una notificación diseñada para ser enviada cuando el valor de cpuUsage supera el de cpuThreshold. Esta notificación incluye las siguientes variables (varBinds): cpuUsage, cpuThreshold, managerEmail y dateAndTime.

2.2 MYAGENT-MIB

Después, para realizar la MIB personalizada, MYAGENT-MIB, se nos ha proporcionado un número IANA de una empresa específica, en este caso el número es el 28308, que corresponde con la empresa "Zaragoza Network Management Research Group". Una vez tenemos el IANA de esta empresa, hemos construido la MIB desde el OID Enterprises.28308.1, situando todos los objetos y notificaciones posteriores dentro de este espacio de nombres único. Para revisar que nuestro árbol estaba bien hecho le preguntamos a la IA Gemini, si estaba bien diseñado, a lo cual, nos dijo que estaba perfecta.

2.3 ARCHIVO JSON

En siguiente lugar, hemos realizado un archivo JSON llamado "mib_state.json", que sirve como almacén de datos persistente para el agente. De esta forma el agente retiene los valores de configuración como manager, managerEmail y cpuThreshold, incluso después de un reinicio. Esto elimina la necesidad de reconfigurar el agente cada vez que se inicia, lo que lo hace más robusto y práctico para un entorno de producción.

La estructura del archivo JSON se ha diseñado para reflejar directamente los objetos escalares definidos en la MIB. Además, este archivo también lo verificamos con Gemini.

2.4 MINI_AGENT.PY

Una vez hecha la mib personalizada y el archivo JSON, hemos llevado a cabo un script en python que va a hacer de agente snmp de la MIB creada. Como este agente debe de trabajar sobre la librería pysnmp v7.1.4, hemos ido tomando como referencia las partes del código proporcionadas por el enunciado, que están incompletos ya que usan una versión de pysnmp anterior, y los hemos ido traduciendo con Gemini y ChatGPT a base de probar el código generado por la IA y corregir los errores que estos podrían tener. Una vez que vimos que el script de python se ejecutaba sin problemas, estudiamos y limpiamos el código completo para ver que no hubiera nada raro o que algo no concordase con los objetivos del enunciado del trabajo.

2.5 TEST.PY

Finalmente, hemos realizado el script “test.py” para verificar el funcionamiento de nuestro agente. Para llevar a cabo las diferentes pruebas que se nos pedían en el enunciado, hemos pedido ayuda también a Gemini, y hemos seguido el mismo proceso que hemos usado para realizar el agente. Una vez vimos que las pruebas se llevaban a cabo con éxito, limpiamos el código y le añadimos una interfaz de usuario para poder realizar las pruebas de manera individual y ordenada.